

GRADUATION PROJECT PROPOSAL

TaxiStream Intelligence

Real-Time NYC Taxi Trip Duration Prediction Platform

Team Members

Name	GitHub
Mustafa Ehab Aql	https://github.com/Eng-MostafaEhab
Youssef Mohamed Eldwaltly	https://github.com/aldwaltly
Abdallah Mahmoud	https://github.com/Asoussa
Mina Maged	https://github.com/MinaMaged88

Group: S26-B3-Big Data-G3-E

Department: Computer Science / Data Engineering

Faculty: Faculty of Computers and Artificial Intelligence

Date: July 2026

Domain: Smart Mobility • Big Data Engineering • Streaming Analytics • Machine Learning

Table of Contents

- Project Overview & Concept
- Tools & Technologies Used
- Project Workflow & Process
 - 3.1 Storage & Batch Foundation
 - 3.2 Stream Simulation & Ingestion
 - 3.3 Real-Time Processing & ML Inference
 - 3.4 Serving & Visualization
- System Architecture
- Machine Learning Pipeline & Strategy
- Functional & Non-Functional Requirements
- Challenges & Mitigation Strategies
- Project Deliverables

1. Project Overview & Concept

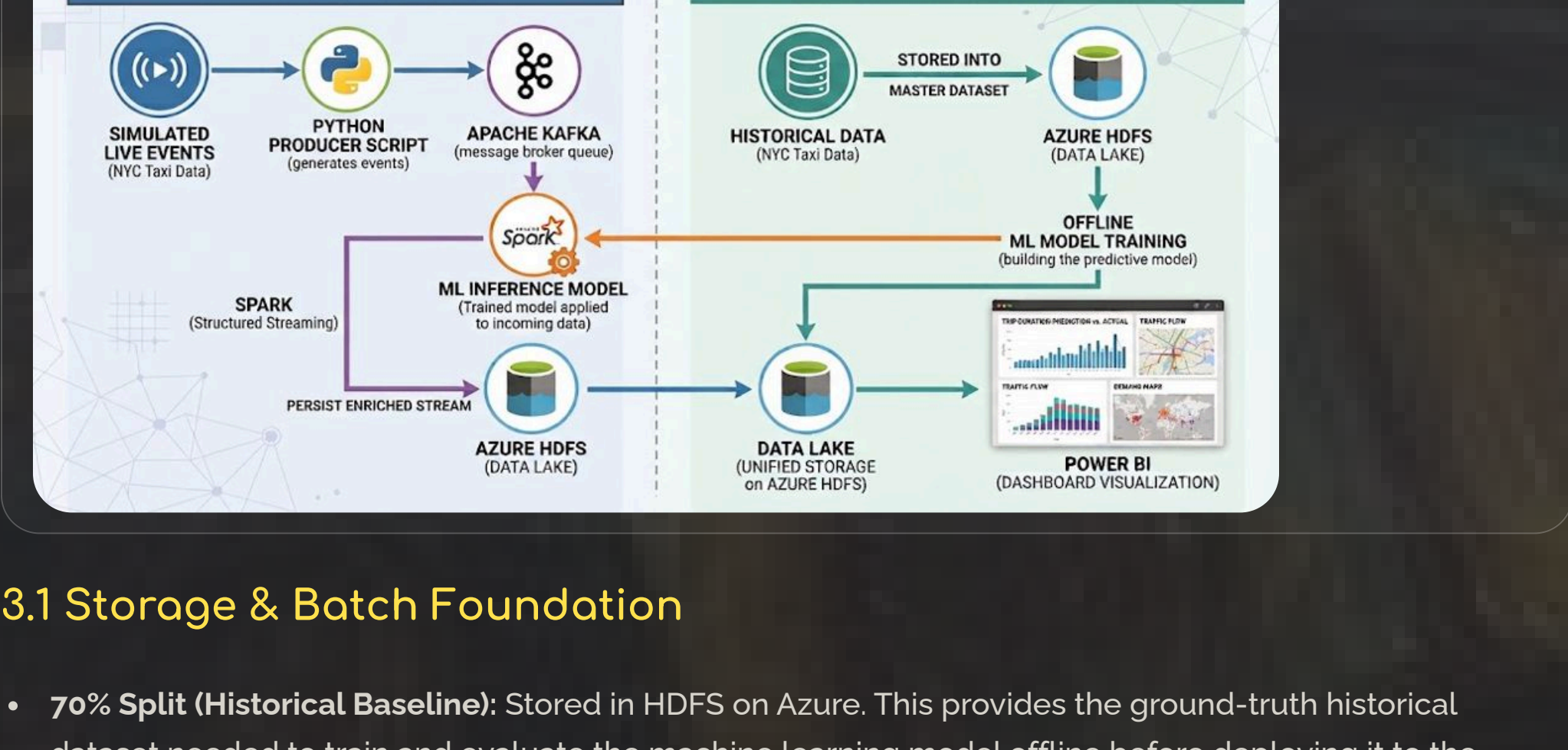
TaxiStream Intelligence is an end-to-end Big Data and Machine Learning solution designed to process, analyze, and predict New York City taxi trip durations in real time.

Built on a **Lambda Architecture** framework, the system seamlessly handles both historical batch data and live stream data. Historical batch data is processed offline to establish baseline analytics and train predictive machine learning models. Concurrently, live stream data simulating real-time taxi trips is ingested and scored in low-latency micro-batches. By unifying historical batch storage with real-time streaming inference, TaxiStream Intelligence delivers actionable mobility insights through interactive dashboards.

2. Tools & Technologies Used

Category	Tool / Technology	Role in Project
Storage & Cloud	HDFS on Azure Cloud	Serves as the central Data Lake for both historical data (70% split) and persisting enriched streamed results.
Data Ingestion	Python (Producer Script)	Simulates real-time IoT taxi meters by streaming raw trip events sequentially.
Message Broker	Apache Kafka	Buffers high-throughput streaming events and ensures fault-tolerant delivery.
Processing & ML	Apache Spark (Streaming & MLlib)	Consumes streams, extracts features, and runs real-time inference using the trained ML model.
Visualization	Power BI	Renders dynamic dashboards comparing predicted vs. actual durations and spatial trip patterns.
Containerization	Docker	Packages the ingestion tools and producer runtime into portable, reproducible container to prevent environment drift.

3. Project Workflow & Process



3.1 Storage & Batch Foundation

- 70% Split (Historical Baseline):** Stored in HDFS on Azure. This provides the ground-truth historical dataset needed to train and evaluate the machine learning model offline before deploying it to the streaming environment.

3.2 Stream Simulation & Ingestion

- Python Producer:** Simulates real-time event generation by reading the remaining 30% of the raw data line-by-line or in micro-batches and sending it directly to Kafka topics.
- Apache Kafka:** Acts as the high-throughput, fault-tolerant message broker, buffering stream traffic and decoupling the data producer from the stream processing engine.

3.3 Real-Time Processing & ML Inference

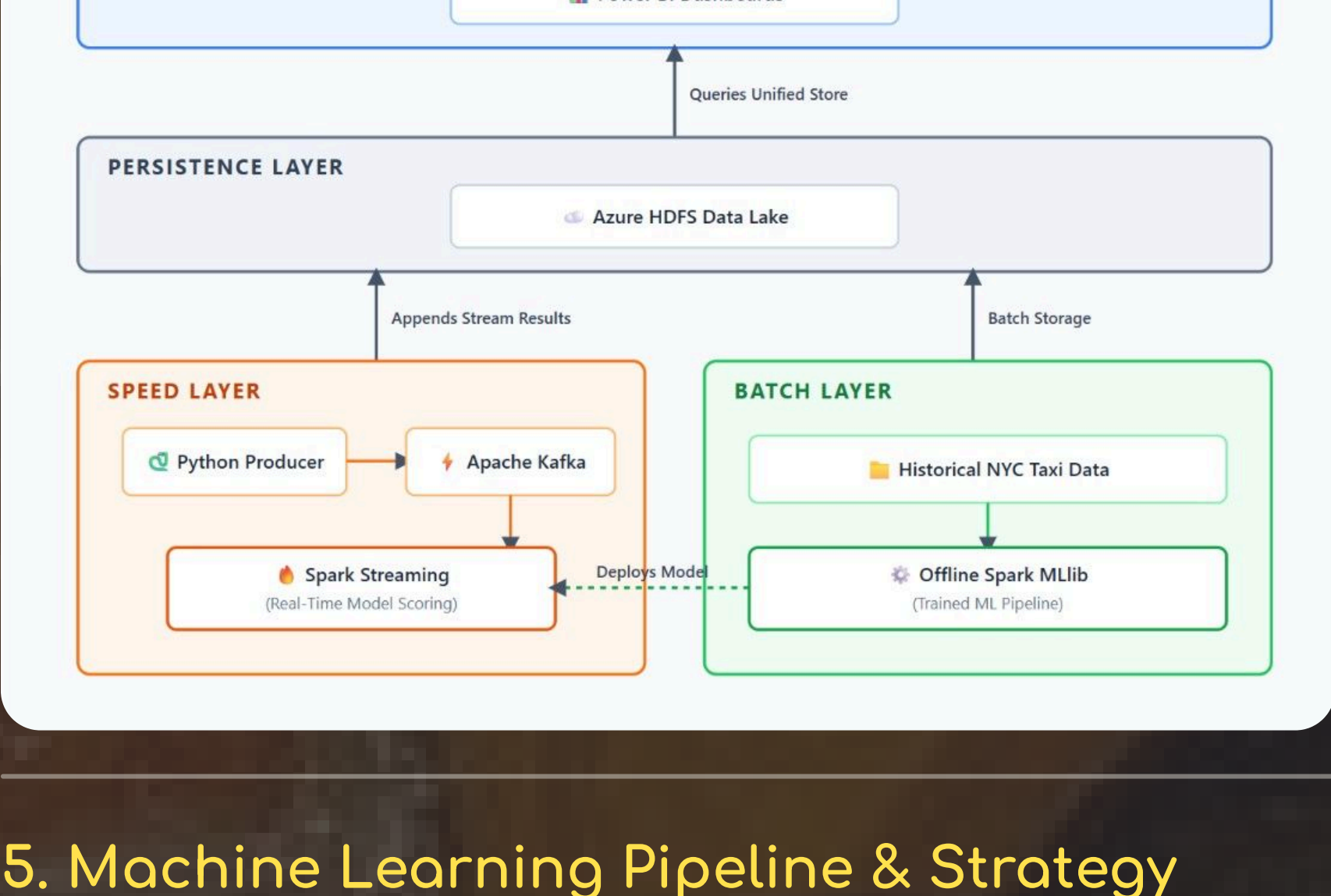
- Spark Streaming (Structured Streaming):** Consumes events from Kafka in real time, applies geospatial and temporal feature transformations, and passes the incoming data through the pre-trained ML model to predict trip_duration.
- Sink to HDFS:** Writes the enriched stream (raw features + predicted duration) back to HDFS on Azure, appending it to the unified historical data lake.

3.4 Serving & Visualization

- Power BI:** Connects directly to HDFS on Azure to render dynamic dashboards comparing predicted vs. actual trip durations, driver bottlenecks, and peak traffic locations.

4. System Architecture

The project architecture strictly follows the **Lambda Architecture** pattern, splitting processing into a Batch Layer, a Speed Layer, and a unified Serving Layer:



5. Machine Learning Pipeline & Strategy

01	Feature Extraction	02	Offline Model Training
Spatial features (pickup/dropoff latitude & longitude, distance via Haversine formula) and temporal features (pickup hour, day of the week, holiday indicators).		Trained using PySpark MLlib on historical data stored in Azure HDFS. Regression models evaluated include Random Forest Regressor and Gradient Boosted Trees (GBT).	
03	Model Serialization	04	Streaming Scoring
The trained pipeline model is serialized and exported to Azure HDFS.		Spark Structured Streaming loads the model pipeline on startup and evaluates incoming micro-batch vectors on the fly.	

6. Functional & Non-Functional Requirements

6.1 Functional Requirements

- Ingest static trip records into Azure HDFS.
- Simulate live IoT taxi meter events using a Python producer sending data to Apache Kafka.
- Execute low-latency micro-batch transformations and real-time model scoring using Spark Structured Streaming.
- Store unified enriched records (input variables + predictions) back into Azure HDFS.
- Display operational metrics and predictions using Power BI dashboards.

6.2 Non-Functional Requirements

Low Latency Inference on streaming records must complete in under 1 second per micro-batch.	Scalability System components (Kafka partitions and Spark executors) must scale horizontally.	Fault Tolerance Spark checkpointing and Kafka message replication to prevent data loss.
---	---	---

7. Challenges & Mitigation Strategies

Challenge	Expected Impact	Mitigation Strategy
Stream Backpressure	Traffic spikes causing latency spikes in Spark.	Set maxOffsetsPerTrigger in Spark to cap incoming micro-batch size.
Model Drift	Predictions become stale as traffic patterns evolve.	Schedule regular retraining pipelines using updated batch logs in Azure HDFS.
Cloud I/O Latency	Frequent small writes slowing down HDFS persistence.	Compact micro-batches and save outputs in optimized Parquet format.

8. Project Deliverables

1	Pipeline Source Code Complete code base for the Python Kafka Producer, Spark Streaming consumer, and feature engineering transformations.
2	Trained ML Models Serialized machine learning models stored in Azure HDFS.
3	Azure Data Lake Setup Structured HDFS directory layout holding historical and enriched stream data.
4	Power BI Dashboards Fully interactive .pbix report files visualizing traffic flow and trip duration analytics.
5	Technical Documentation Comprehensive architecture guide and setup instructions.